Programming Project - Group 22

Jurij Cerar - 12327958, Peter Szasz - 12333448

January 13, 2025

Contents

1 Introduction **3**

2 Exercise 1: Benchmark **4**

3 Exercise 2: Sequential Skip List **6**

4 Exercise 3: Concurrent Skip List with Global Lock **9**

5 Exercise 4: Fine-Grained Lazy Skip List **10**

6 Exercise 5: Lock-Free Skip List **13**

7 Speedup of Skip List Implementations **17**

8 Benchmarking Workflow **18**

1 Introduction

The Advanced Multiprocessor Programming (AMP) project explores the design, implementation, and performance benchmarking of concurrent skip lists. Skip lists are probabilistic data structures providing efficient O(log n) operations for search, insert, and delete operations [1]. This project encompasses the development of a sequential skip list as a performance baseline, followed by three concurrent implementations: a globally-locked version, a fine-grained lazy skip list, and a lock-free skip list. Each implementation is benchmarked under varying thread counts, operation mixes, and key range configurations to evaluate throughput and scalability.

The results highlight the trade-offs between synchronization methods and concurrency control. Through detailed performance analysis, the project demonstrates how lock granularity and lock-free techniques influence operation success rates and system throughput. The benchmarks provide insights into the behavior of concurrent skip lists under contention and high concurrency, guiding future optimizations for such data structures in parallel environments.

2 Exercise 1: Benchmark

Objective

The goal of Exercise 1 is to implement a benchmark for various skip lists. This task involves designing the structure, ensuring correctness in concurrent operations, and measuring performance metrics under various configurations to evaluate the behavior of the skip list implementation.

Implementation

Benchmark Overview

It supports configurable parameters such as:

- **Thread Count:** Number of threads participating in concurrent operations.
- **Operation Distribution:** Percentage of operations dedicated to insert, delete, and contains.
- **Key Range:** Range of keys used for operations.
- **Strategies:** Strategies for key generation (RANDOM, UNIQUE, DETERMINISTIC).
- **Prefill Count:** Number of keys pre-inserted into the skip list before benchmarking.
- **Key Range Type:** Distribution of key ranges across threads (DISJOINT, PER_THREAD, COMMON).

Skip List Structure

The skip list supports three primary operations:

1. **Insert:** Adds a key to the skip list if it does not already exist.
2. **Erase:** Removes a key from the skip list if it exists.
3. **Contains:** Checks if a key exists in the skip list.

Each operation uses locks or lock-free techniques to ensure correctness in a concurrent environment.

Special Configuration

1. **Key strategy:** the user can specify what kind of selection strategy to use:
    - **Random:** each key is randomly generated from the given range.
    - **Unique:** each key is randomly generated from the given range, but it is also unique (no repetition).
    - **Successive:** keys are incremented by 1 up to upper limit of range.

2. **Range type:** the user can specify what kind of range type is to be used:
   - **Disjoint:** each thread operates on a completely separate range of keys that do not overlap with other threads.
   - **Per thread:** each thread gets its own distinct key range, but the ranges might not be entirely disjoint.
   - **Common:** all threads operate on the same range of keys, leading to shared access to the same data.

Validation

Validation is implemented to ensure correctness:

$$\text{Expected Count} = \text{Successful Prefill} + \text{Successful Inserts} - \text{Successful Deletes}$$

The `get_element_count` function computes the actual count of elements, which is then compared with the expected count.

In addition, the validation process ensures correct deletion behavior by performing the following checks:

- If a test key is already present in the skip list, the validation verifies that the key can be deleted exactly once and cannot be deleted again.
- If the test key is not present, it is inserted into the skip list, then deleted, and an additional deletion attempt is made to confirm that it cannot be deleted more than once.

The combined checks ensure that the skip list maintains its integrity and prevents invalid deletion behavior. Any discrepancies in the expected and actual counts or in deletion behavior are flagged as validation failures, with appropriate error messages printed to aid debugging.

Chapter 8 describes the manual run of the benchmark with custom configuration.

3 Exercise 2: Sequential Skip List

Objective

The objective of Exercise 2 is to implement a sequential skip list and analyze its performance as a baseline. The skip list supports basic operations such as `insert`, `delete`, and `contains`. The benchmarks measure performance under different operation mixes and validate the correctness of the implementation.

Implementation

The sequential skip list is implemented using a probabilistic multi-level structure [1]. Its features include:

- **Node Structure:** Each node in the skip list is represented by a structure that contains the following:
  - A `key`, which serves as the unique identifier for the node.
  - A `forward` array of pointers, which points to nodes at each level, enabling efficient traversal and search operations.

  The `create_node` function dynamically allocates memory for a node, initializes the key, and sets all forward pointers to `NULL`.

- **Multi-Level Structure:** The skip list is built with multiple levels to optimize search operations. Nodes at higher levels provide shortcuts for traversal, reducing the search complexity to `O(log n)` on average.

- **Random Level Generation:** The level of a new node is determined probabilistically to balance the structure and maintain efficiency. The `random_level` function generates a random level for a new node based on a probability constant `PROBABILITY`. Nodes with higher levels are less frequent, ensuring a balanced skip list structure.

- **Insert Operation:** The `insert` function locates the appropriate position for a key by traversing the skip list from the highest level down to level 0. If the key already exists, the function returns `false`. Otherwise, it creates a new node with a randomly generated level and updates the forward pointers to include the new node.

- **Delete Operation:** The `erase` function removes a key from the skip list by first locating it through the forward pointers. If the key exists, the forward pointers of the previous nodes are updated to bypass the node being deleted. Memory allocated for the node is freed.

- **Search Operation:** The `contains` function checks whether a given key exists in the skip list. It traverses the list from the highest level down to level 0, using the forward pointers to navigate efficiently.

- **Element Count:** The skip list maintains an `element_count` field, which is incremented during successful insertions and decremented during deletions.

- **Memory Management:** The `free_skiplist` function ensures that all dynamically allocated memory is released. It traverses the list, frees the memory for each node's forward pointers, and then frees the node itself.

Results

Benchmarks were conducted using two operation mixes:

- **10-10-80:** Insert (10%), Delete (10%), Contains (80%)
- **40-40-20:** Insert (40%), Delete (40%), Contains (20%)

The plot (Figure 1) shows the throughput for these mixes. The mix with 10-10-80 achieves higher throughput due to the dominance of read operations, which are less resource-intensive compared to inserts and deletes.

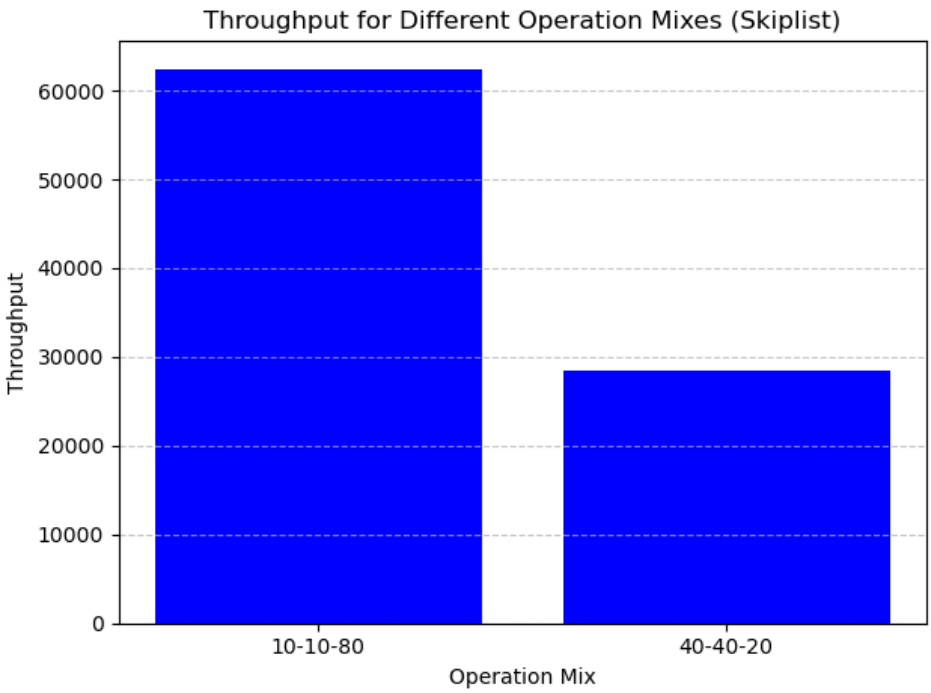

Figure 1: Throughput for Sequential Skip List with Different Operation Mixes

Additionally, the success rates of operations were analyzed to evaluate the effect of different key ranges and operation mixes on the performance of the sequential skip list. The results, as shown in Figure 2, highlight the impact of key range and operation mix on operation success:

- When the key range is reduced, both `delete` and `contains` operations exhibit significantly higher success rates.
- The improvement can be attributed to a smaller search space, which increases the likelihood of encountering valid keys during these operations. This reduces failures caused by attempts to access non-existent keys.

- The success rate also depends on the operation mix, as the ratio of `insert`, `delete`, and `contains` operations determines how many valid keys are available in the skip list. For example, an operation mix with a high proportion of `insert` operations ensures a larger number of keys in the skip list, which improves the success rate for subsequent `delete` and `contains` operations. This might be influenced by the prefilled items too.

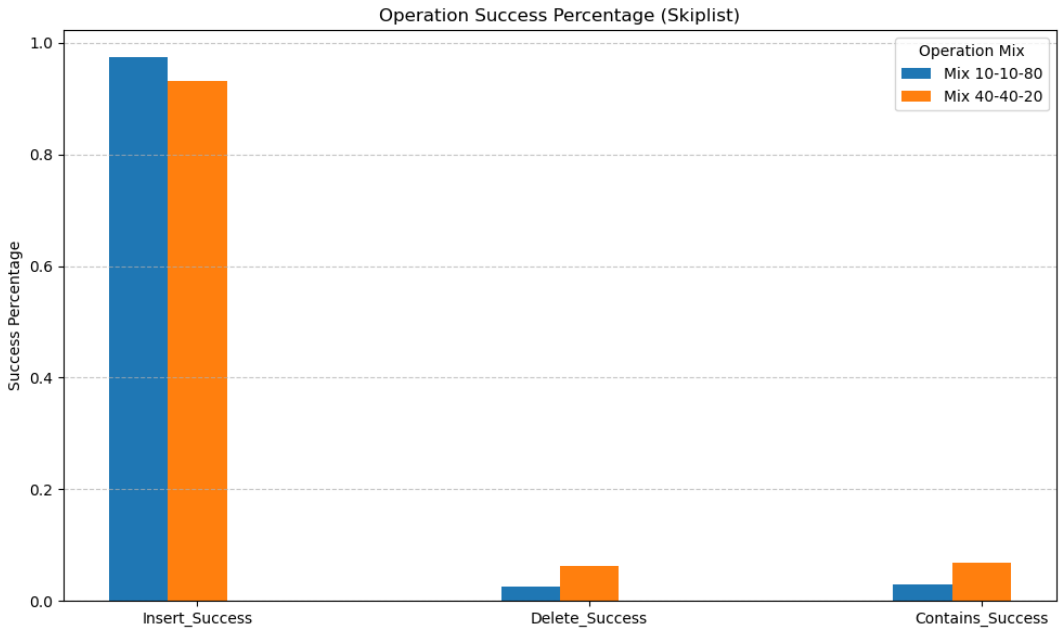

Figure 2: Operation Success Percentage for Sequential Skip List

4 Exercise 3: Concurrent Skip List with Global Lock

Objective

The objective of Exercise 3 is to implement a concurrent skip list using a global lock for synchronization. Benchmarks measure performance under increasing thread counts to evaluate the scalability and contention effects.

Implementation

The skip list employs a global lock to ensure mutual exclusion during operations. This simplifies synchronization but introduces contention as the number of threads increases. Benchmarks were conducted with thread counts ranging from 1 to 64.

Results

The plot (Figure 3) illustrates the throughput vs. thread count for the globally-locked skip list. Key observations include:

- **1 Thread:** The throughput is much higher than with the sequential skip list.
- **2 Threads:** Performance improves due to parallelism.
- **4 to 64 Threads:** Performance becomes worse but stabilizes as contention limits scalability.

The use of a global lock introduces bottlenecks, particularly under high contention.

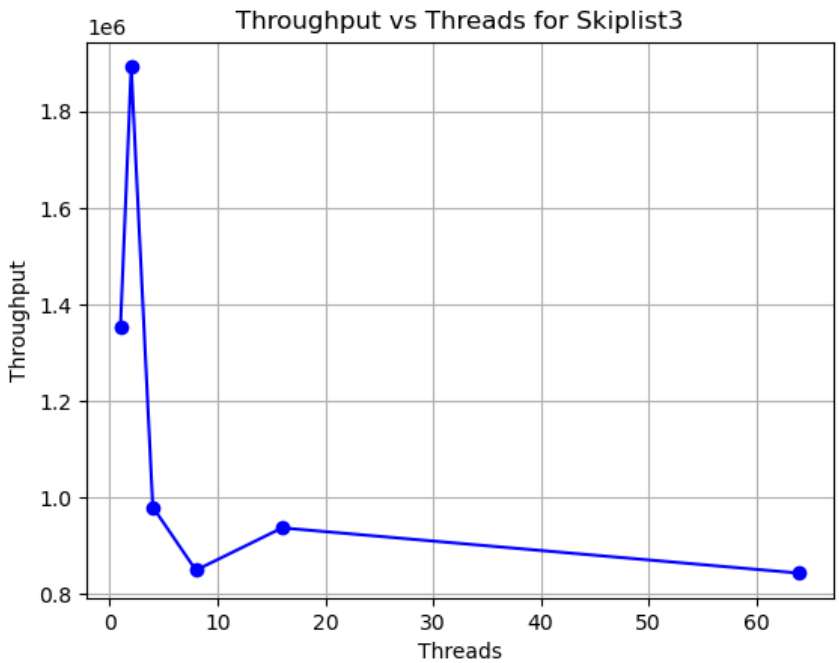

Figure 3: Throughput vs. Threads for Global Lock Skip List (op. mix 10-10-80)

5 Exercise 4: Fine-Grained Lazy Skip List

Objective

The goal of Exercise 4 is to implement a fine-grained lazy skip list with thread-safe operations. Benchmarks evaluate performance under varying thread counts and key range configurations (COMMON and DISJOINT).

Implementation

The lazy skip list employs fine-grained locking, where each node is locked individually during operations [1]. This minimizes contention by allowing concurrent access to disjoint parts of the structure. Key features include:

- Optimistic locking for `contains` operations.
- Fine-grained locking for `insert` and `delete` operations to ensure correctness.

Implementation Enhancements

To ensure thread-safe updates, the implementation uses `#pragma omp atomic` directives for incrementing the `inserted_count` and `deleted_count` counters. These atomic counters help validate the consistency of the skip list by maintaining accurate counts of inserted and deleted elements across concurrent operations.

Additionally, a helper function, `unlockLevels`, is used to efficiently release locks on predecessor nodes after operations such as `insert` and `erase`. This function ensures that locks are properly managed, preventing deadlocks and improving concurrency.

Results

Benchmarks were conducted with thread counts from 1 to 64 and key ranges configured as COMMON and DISJOINT. The results are presented in two parts: performance analysis and success rate analysis. The plots are based on the operation mix 10-10-80. The mix 40-40-20, though not explicitly shown, demonstrates similar trends in terms of performance improvement, however, at a slower rate, as explained below. The plots are populated with the benchmark results, the difference between these results and the results from Exercise 2 and Exercise 3 is the number of experiments. The given configuration was tested 3 times to visualize the potential deflections.

Performance Analysis

- **COMMON Key Range:** Throughput increases with thread count, as shown in Figure 4. However, the performance gains begin to taper off at higher thread counts. This is because all threads operate on the same set of keys, leading to frequent conflicts during insert and delete operations. These conflicts cause threads to wait for locks, limiting the scalability of throughput despite the increase in parallelism.

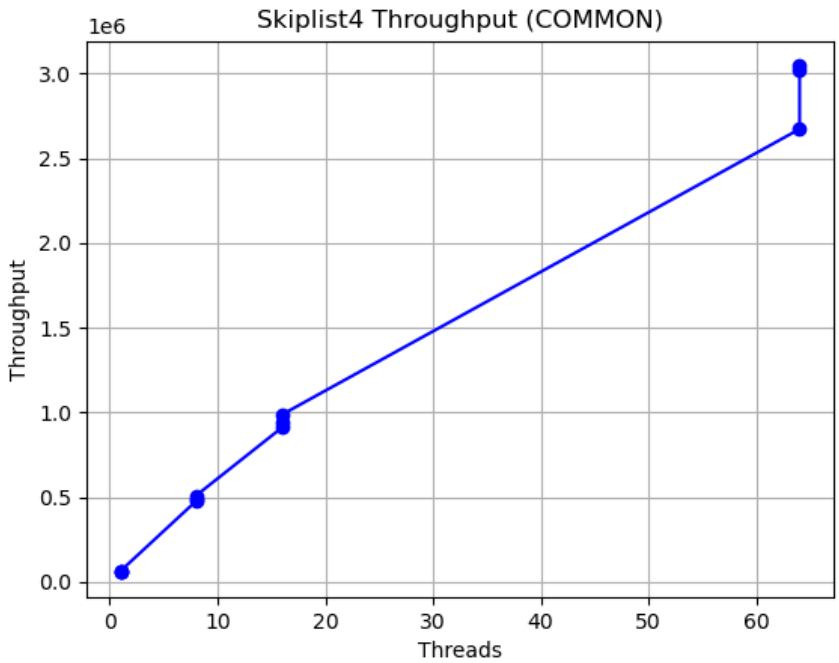

Figure 4: Throughput vs. Threads for Fine-Grained Lazy Skip List (COMMON, op. mix 10-10-80)

- **DISJOINT Key Range:** Throughput steadily improves with increasing thread count, as shown in Figure 5. Similarly to the configuration with the common key range, the performance gains begin to flatten at higher thread counts, however, it reaches the peak slower than with the common key range. In this configuration, each thread operates on a unique set of keys, eliminating contention between threads. This independence allows threads to execute their operations concurrently without interference, leading to more consistent performance gains across all thread counts.

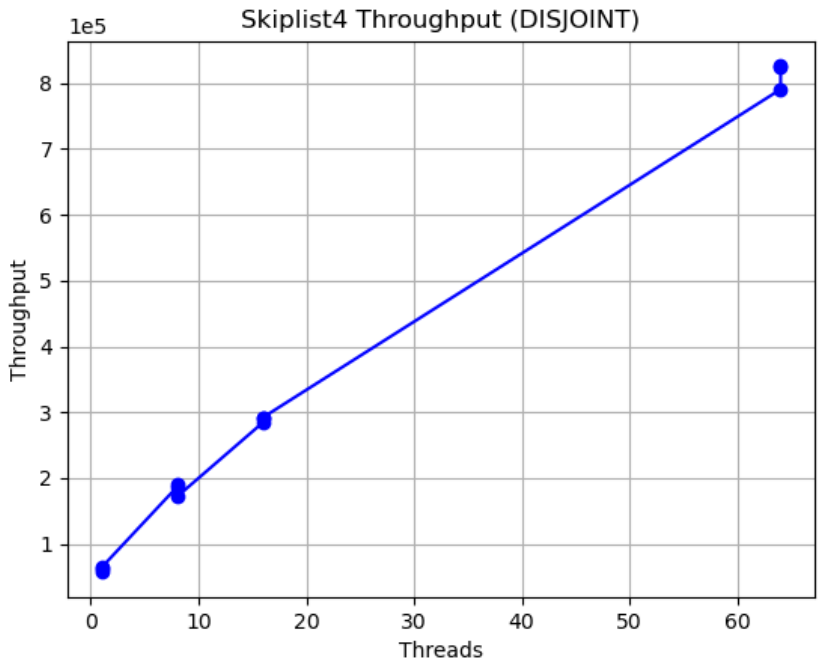

Figure 5: Throughput vs. Threads for Fine-Grained Lazy Skip List (DISJOINT, op. mix 10-10-80)

The 10-10-80 operation mix improves throughput more rapidly compared to the 40-40-20 mix. This is because the mix 10-10-80 is dominated by `contains` operations, which are less resource-intensive and do not modify the data structure. In contrast, the mix 40-40-20 has a higher proportion of `insert` and `delete` operations, which involve locking and structural modifications, thereby introducing additional overhead and slowing down performance improvement.

Success Rate Analysis

The success rates of operations under COMMON and DISJOINT key ranges reveal the impact of contention. Figure 6 shows the percentage of successful operations for both configurations with 64 threads.

- **COMMON Key Range:** The success rate decreases as the thread count increases. This is a direct consequence of contention, as multiple threads compete to modify the same keys. The frequent locking and unlocking required to resolve conflicts further reduces the likelihood of successful insert or delete operations.
- **DISJOINT Key Range:** The success rate remains significantly higher across all thread counts. Since threads operate on distinct key ranges, there is no contention for shared keys. This independence not only increases the overall success rate but also reduces the time spent waiting for locks, improving system responsiveness.

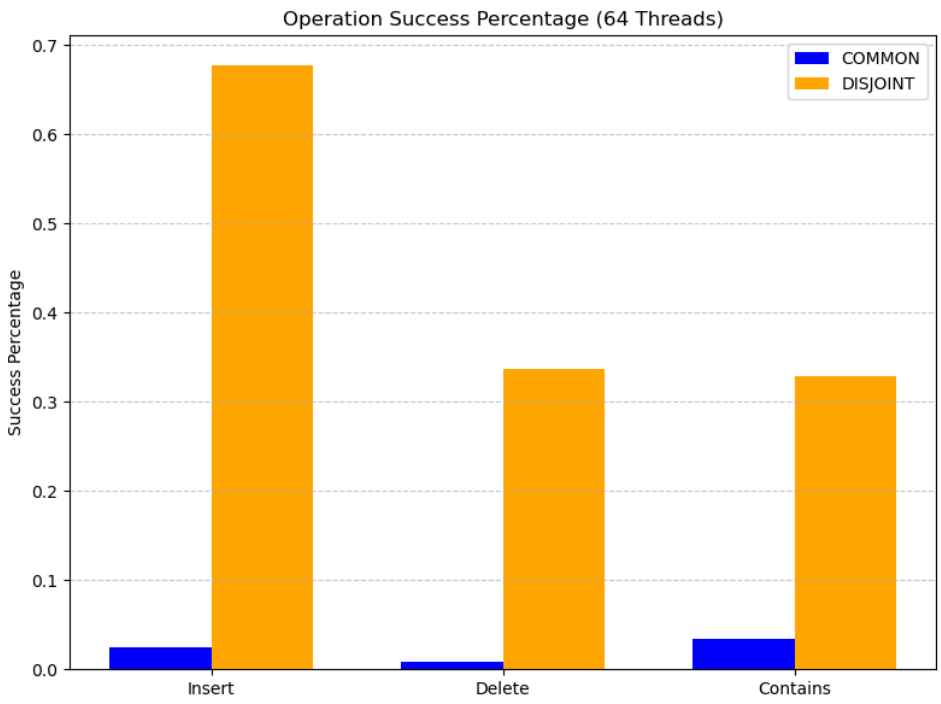

Figure 6: Success rate of operations with different key ranges (op. mix 10-10-80)

The tests with the other thread counts reveal a similar pattern in the success rate of the operations in relation to the disjoint and common key ranges.

6 Exercise 5: Lock-Free Skip List

Objective

The objective of Exercise 5 is to implement a lock-free skip list, leveraging atomic operations and compare-and-swap (CAS) to ensure thread-safe operations without requiring locks. Benchmarks evaluate its performance and success rates under the same configurations as the fine-grained lazy skip list.

Implementation

The lock-free skip list uses atomic markable references to manage node pointers and marking bits. The key advantage of this approach is the elimination of locks, which reduces contention and avoids deadlocks. Operations such as `insert`, `delete`, and `contains` are implemented using CAS to atomically update pointers while ensuring consistency [2, 3].

- **Marking Bits for Logical Deletion:** The lock-free skip list employs marking bits to manage logical deletion of nodes. Each pointer in a node is represented using an atomic markable reference, which includes a bit to indicate whether the pointer is marked for deletion. Helper functions, such as `get_flag` and `set_flag`, are used to extract and manipulate these marking bits. This mechanism ensures that nodes are logically removed without disrupting concurrent operations on other parts of the list.

- **Atomic Operations with Compare-And-Swap (CAS):** The lock-free skip list relies on the `compareAndSet` function to perform atomic updates to pointers. This function combines the ability to update both the pointer and its marking bit in a single operation, ensuring consistency in the presence of concurrent modifications. Operations like `insert`, `delete`, and `find` all utilize CAS to traverse and modify the skip list safely. Additionally, the `attemptMark` function is used to atomically mark a pointer, enabling logical deletion during the `erase` operation.

Results

Benchmarks for the lock-free skip list were conducted with thread counts from 1 to 64, using COMMON and DISJOINT key ranges. Similar to Exercise 4, the results are divided into performance analysis and success rate analysis. The plots are populated with the benchmark results, the difference between these results and the results from Exercise 2 and Exercise 3 is the number of experiments. The given configuration was tested 3 times to visualize the potential deflections similarly to Exercise 4.

Performance Analysis

- **COMMON Key Range:** Throughput improves more rapidly with increasing thread counts, as shown in Figure 7. However, contention still affects performance, especially at higher thread counts, as threads compete to update the same set of keys. This contention limits the scalability of the lock-free implementation, though it performs slightly better than its fine-grained counterpart under similar conditions.

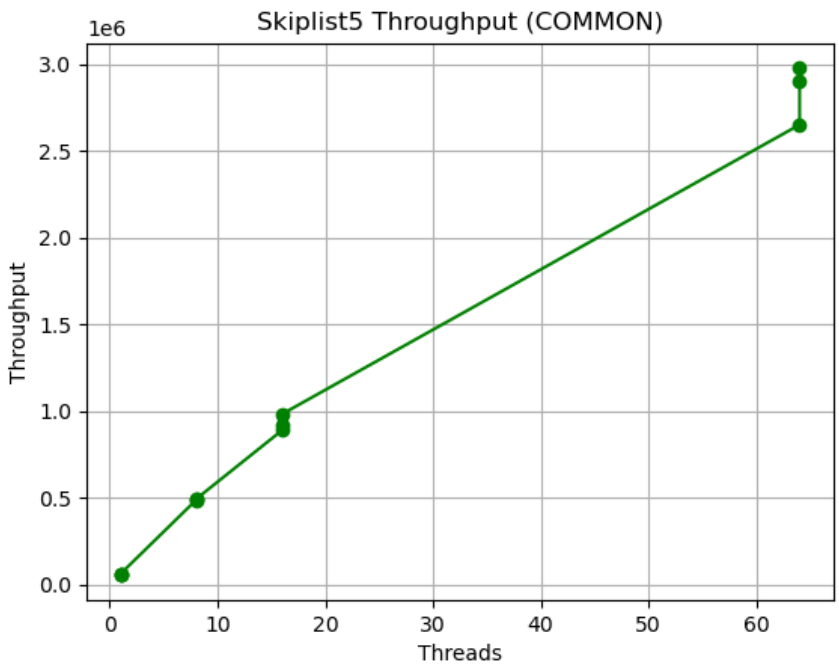

Figure 7: Throughput vs. Threads for Lock-Free Skip List (COMMON, op. mix 10-10-80)

- **DISJOINT Key Range:** Throughput steadily improves across all thread counts,

as shown in Figure 8. The disjoint key range ensures that threads operate independently, minimizing contention and allowing the lock-free implementation to take full advantage of parallelism. This results in smoother and more predictable performance gains compared to the COMMON key range.

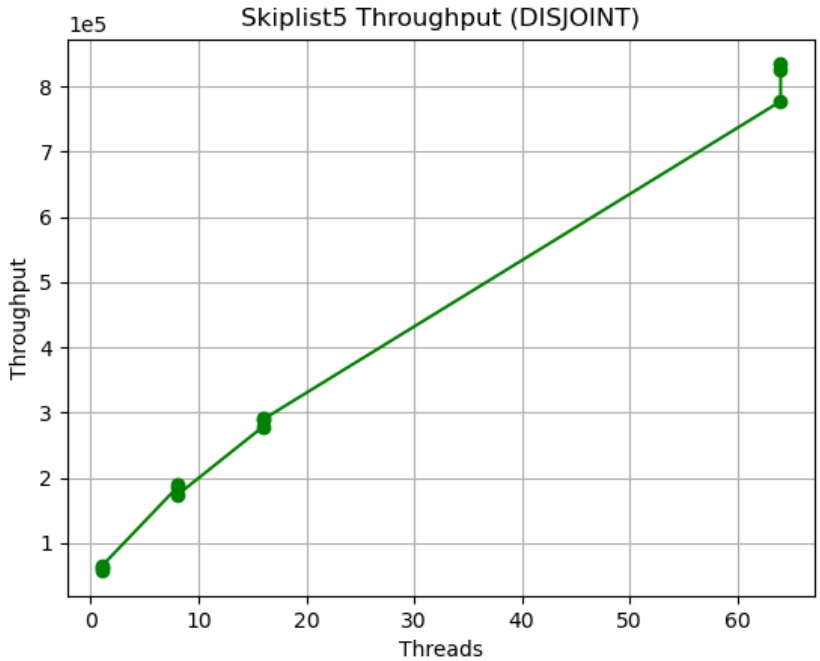

Figure 8: Throughput vs. Threads for Lock-Free Skip List (DISJOINT, op. mix 10-10-80)

The operation mix 10-10-80 again shows more rapid improvement in throughput compared to 40-40-20. The dominance of read operations (`contains`) reduces contention and computational overhead, allowing the system to process more operations per unit of time. In contrast, the mix 40-40-20 involves more structural changes, which are inherently slower due to the additional synchronization required.

Success Rate Analysis

Figure 9 shows the success rates for the COMMON and DISJOINT key ranges. The results are similar to the ones in Exercise 4 with the same configurations (64 threads).

- **COMMON Key Range:** The success rate decreases with increasing thread counts due to contention for shared keys. Although lock-free operations reduce waiting times, the fundamental issue of conflicting updates remains, limiting the success rate.
- **DISJOINT Key Range:** Success rates remain high across all thread counts, as threads operate on separate key ranges. This eliminates conflicts and ensures that operations complete successfully without interference.

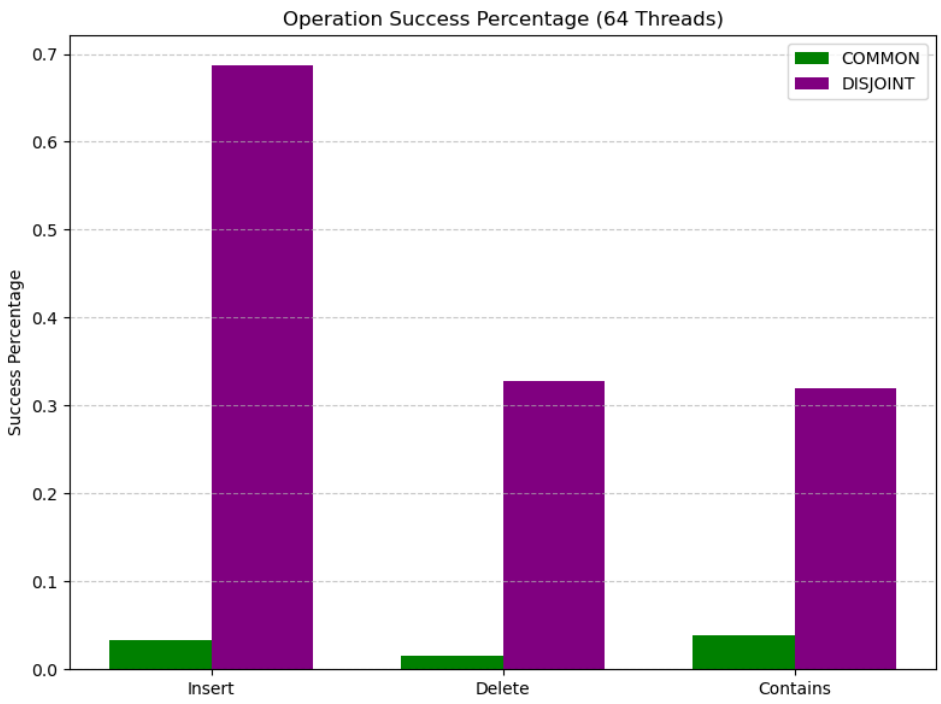

Figure 9: Success rate of operations with different key ranges (Lock-Free Skip List, op. mix 10-10-80)

The relation of the success rates between disjoint and common key ranges behave similar to the results observed in Exercise 4 when tested with a different thread count.

7 Speedup of Skip List Implementations

The performance of the concurrent skip list implementations significantly surpasses that of the sequential skip list under multi-threaded workloads, despite varying degrees of scalability.

Global Lock Skip List

The global lock skip list demonstrates a substantial speedup compared to the sequential skip list, particularly for lower thread counts. This is attributed to the ability of multiple threads to operate concurrently, even though they are synchronized using a single global lock. However, as contention increases with higher thread counts, performance begins to degrade, limiting its scalability. The results aligned with our expectations, demonstrating improved performance up to a peak point, after which it begins to decline.

Fine-Grained Lazy Skip List

The fine-grained lazy skip list achieves a considerable speedup over both the sequential skip list and the global lock skip list. By employing individual locks for each node, this implementation minimizes contention and allows threads to operate more independently. The improved scalability results in consistently higher throughput, particularly under configurations with DISJOINT key ranges, where contention is entirely eliminated. We anticipated better results compared to Exercises 2 and 3, and this expectation was validated through the tests conducted on the lazy skip list.

Lock-Free Skip List

The lock-free skip list provides the highest speedup among all implementations, leveraging atomic operations to avoid locks altogether. This approach significantly reduces contention and overhead, leading to smoother performance improvements as thread count increases. The lock-free skip list consistently outperforms the fine-grained lazy skip list, especially under DISJOINT key range configurations, where it fully capitalizes on parallelism without synchronization bottlenecks. Our initial assumption regarding the lock-free skip list performance, when compared to the results from Exercise 4, was surprising. Contrary to expectations, the lock-free implementation did not consistently outperform the fine-grained locking approach. Depending on the thread count and configuration, the two implementations were often closely matched in performance, with no clear winner across all scenarios.

8 Benchmarking Workflow

The benchmarking workflow is automated using the `run_nebula.sh` script, which should be executed from the project directory using the following command:

```
bash run_nebula.sh project.zip small-bench
```

The process is as follows:

1. The script copies the project archive to the Nebula server using `scp`.
2. The project is extracted, compiled, and benchmarks are executed on the server.
3. Benchmark results are copied back to the local machine.
4. The `benchmark.py` script processes the results and generates plots automatically.

Additionally, if manual generation of plots is required, this can be achieved using the following command from the project directory too:

```
make small-plot
```

This command runs the `benchmark.py` script locally, processes the results, and generates the plots in the specified output directory.

This setup ensures reproducibility and simplifies performance evaluation across different configurations, while also allowing flexibility for manual plot generation when needed.

Manual Benchmarking

For users who prefer to run benchmarks manually, the process can be carried out with the following steps:

1. **Navigate to the Source Directory:** Change to the `src` folder in the project directory.
2. **Compile the Skip List Implementation:** Use the `gcc` command to compile the desired skip list implementation. Replace `H-FILE` and `C-FILE` with the appropriate header and source file for the skip list you wish to benchmark. The general command is:

   ```
   gcc -O3 -Wall -Wextra -fopenmp -g \
       -DSKIPLIST_HEADER='"H-FILE"' \
       -o skiplist_benchmark library.c C-FILE
   ```

3. **Example Compilation Command:** To compile the `skiplist3` implementation, the command would be:

   ```
   gcc -O3 -Wall -Wextra -fopenmp -g \
       -DSKIPLIST_HEADER='"skiplist3.h"' \
       -o skiplist_benchmark library.c skiplist3.c
   ```

4. **Run the Benchmark:** Once compiled, execute the `skiplist_benchmark` binary with the desired configuration. For example, the following command runs a benchmark with 1 thread, 1 experiment, and a duration of 1 second, using a key range of 0 to 50,000, and a 10-10-80 operation mix:

```
./skiplist_benchmark --threads 1 --experiments 1 --duration 1 \
    --operations 10 10 80 --key_range 0 50000 --strategy RANDOM \
    --seed 12345 --prefill 0 --range_type COMMON
```

References

[1] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*, pages 329–349. Morgan Kaufmann, 2012.

[2] Oracle. Atomicmarkablereference (java platform se 8), 2018.

[3] The Open Group. stdatomic.h — base definitions for iso c, 2025.